

Gerald Weber

Transaction Processing

How to perform composite updates safely for a remotely and concurrently accessed system.

Concurrency, ACID properties

Example: Flight booking: Concurrency

- An online multi-leg flight booking system (web or app).
- Users submit their orders and expect them to be:
 - Processed as quickly as possible, also meaning that the user gets quick feedback on success/no success.
 - Exactly as requested, in particular complete:
 - Book all legs of the flight, or do nothing.
- Problem **Concurrency**: different clients might interfere
 - System checked that all legs were available
 - but while booking: first flight ok, last flight is full.

This just in....

QTY	TICKET TYPE	PRICE
0 ▼	Allocated Seating ▼	B
1 ▼	Floor Standing	N
0 ▼	Loyalty Meet & Greet Pac...	N
0 ▼	Damn. Early Entry Package	N

Sorry, no results match your search.
Try changing ticket type.

 **Find Resale Tickets**

Please I
Map is
a guide
and mo
any time



tekeiller

@tekeiller



how are GA [@kendricklamar](#) sold out 22 secs after tickets go on sale [@Ticketmaster_NZ](#)?! 🙄 #kingkendrick

12:11 PM - Apr 30, 2018



4



See tekeiller's other Tweets

Solution Proposal

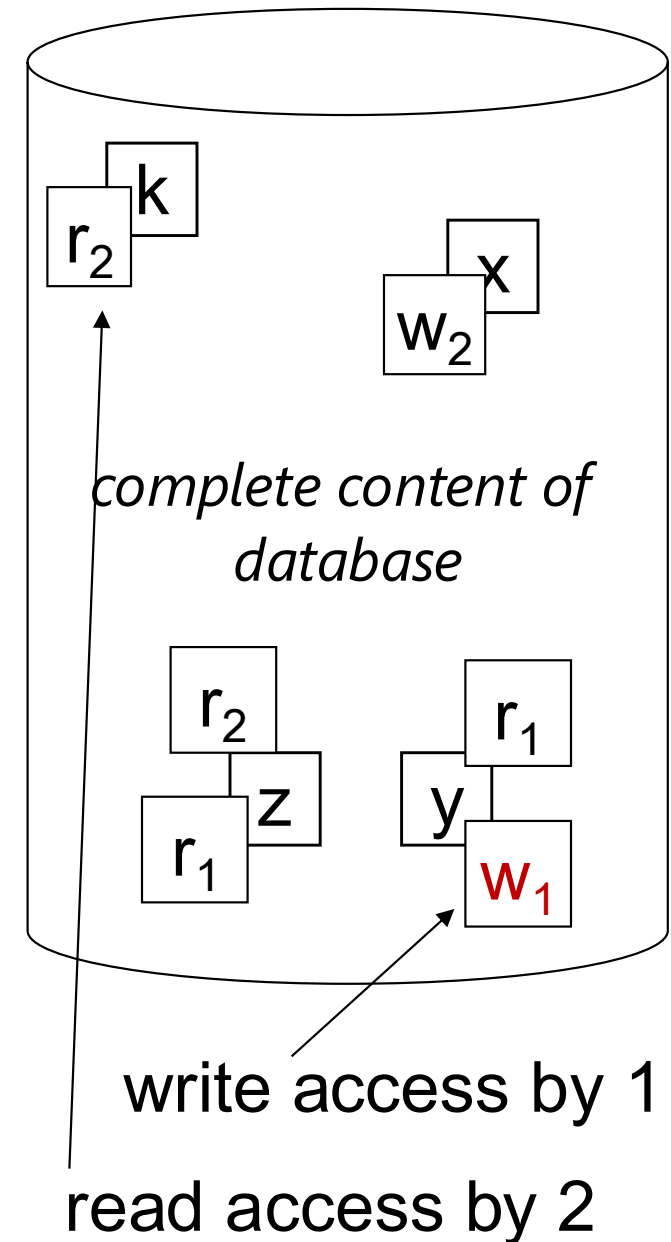
- Simple **Solution** proposal for Concurrency Problem:
 - **DB-wide Mutual exclusion (Mutex)** :
User orders are executed one at a time.
- System again needs to know when user order ends: commit.
- Must know when the next user's order can start.
- Needs again transaction demarcation!
Transaction: A sequence of operations that form a logical unit;
sequence of operations delimited by TA demarcation.
- In mutual exclusion, Transactions can get delayed.

Analysis of Solution Proposal

- **Scalability limits:** DB-wide Mutual exclusion for user orders puts a strict limit on the **throughput** of the system:
 - DB-wide MutEx **serializes** all transactions.
 - **transaction script** needs all parameters beforehand, does not talk to human operator.
 - Assume for sake of argument: each transaction takes on average 0.04 seconds to process on current hardware:
 - Average number of transactions per second for MutEx: ?

Case against DB-wide Mutex: disjoint-access TAs

- **Shared object**: a data object accessed by more than one transaction in a set.
- A set of transactions is **disjoint-access**, if it has no shared object.
- thought experiment: If concurrent transactions **access** random data in a huge database,
 - then often they are disjoint-access.
- Also, if transactions are concerned with unrelated matters:
 - they are probably disjoint-access.



Read/Write Model (abstract Key/Value Store):

- DB storing a single current value per key: A Map.
- Keys are initialized with a null value.
- Two operations:
 - *Read* **r[x]** | `y=db.get(x)` : get value for key x
 - *Write* **w[x]** | `db.put(x,y)` : replace val. for key x with y
- In Java: `db.put(key, val); y=get(key);`
- What is the value of y?

Read/Write Model (abstract Key/Value Store):

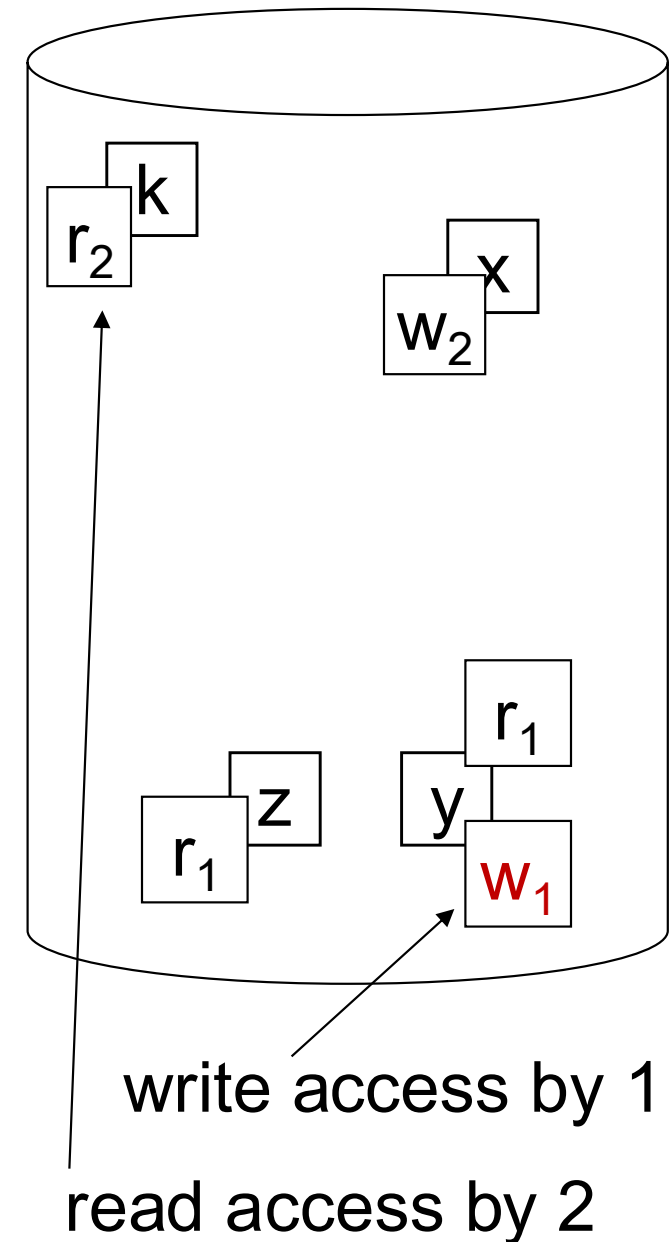
- We focus on read/write[object] for eg a transaction TA1, $r_1[x]$, $w_1[x]$
- *Read* $r_1[x]$ | $y=db.get(x)$: get value for key x
- *Write* $w_1[x]$ | $db.put(x,y)$: repl. val. for key x with y
- *Commit* c_1
- *Abort* a_1

Disjoint-access parallelism (DAP) [e.g. Israeli et al 1994]

- Shared object: a data object accessed by more than one transaction in a set.
- A set of transactions is disjoint-access, if it has no shared object.
- Disjoint-access transactions should not delay each other, ideal:

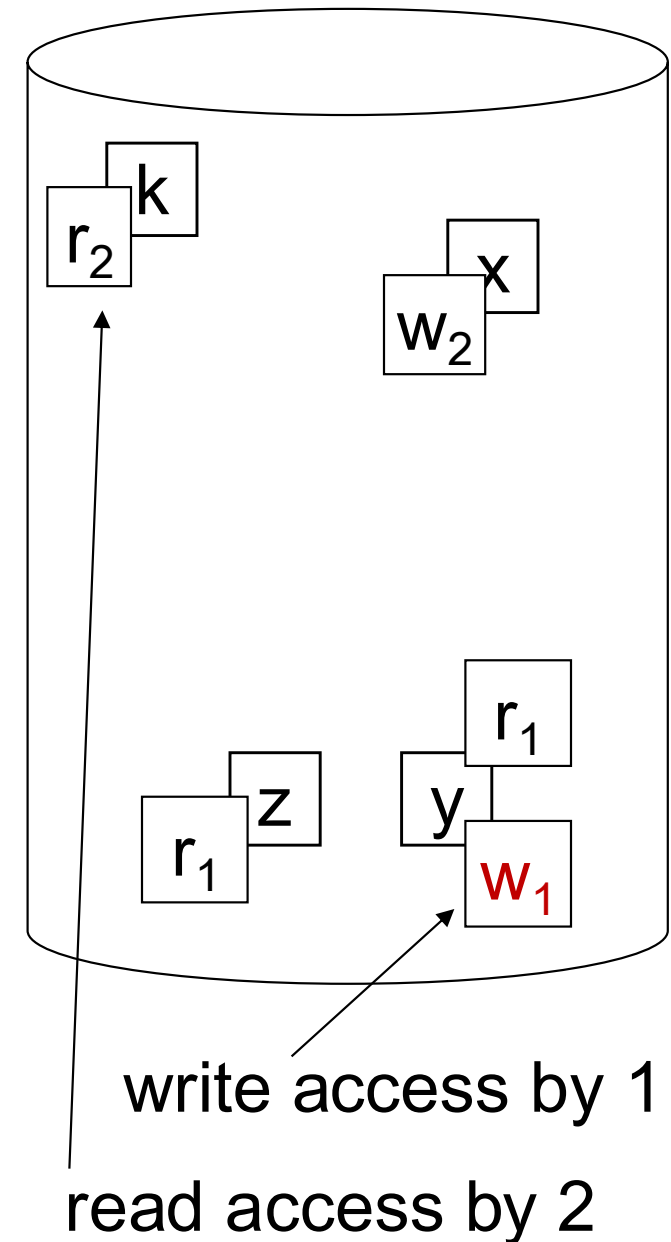
Disjoint-access parallelism (DAP)

A. Israeli and L. Rappoport. Disjoint-access-parallel Implementations of Strong Shared Memory Primitives. In PODC, pages 151–160, 1994.



DB-wide Mutex bad for disjoint-access TAs

- Shared object: a data object accessed by more than one transaction in a set.
- A set of transactions T is disjoint-access, if it has no shared object.
- Principle: disjoint-access transactions do not interfere.
- Hence should not be delayed
- But DB-wide Mutex does delay them:
- DB-wide Mutex is too blunt a tool:
- So: how to detect whether transactions are disjoint-access?



Challenge Transaction processing:

- **Scalability limits:** DB-wide Mutual exclusion user order puts a strict limit on the **throughput** of the system:
- Challenge: Design a solution that offers clients mostly the same interface as for MutEx, but does not unnecessarily delay transactions,
 - does not delay disjoint-access transactions

The ACID properties

- Widely accepted requirements that a transaction manager must meet for the transactions [Gray et al, 1974, 1983]:
 - **Atomicity**: either all of the operations of a transaction are made durable or none of them are.
 - **Consistency**: after the transaction, the database is in a consistent state.
 - **Isolation**: operations in a transaction appear isolated from all other operations. Transactions have a virtual serial view on the system.
 - **Durability**: once the user has been notified of success, the transaction will persist, and not be undone.

ACID atomicity

1. Only if the client has requested commit, write operations in the transaction may become durable.

Atomicity protects against client failures that lead to incomplete transaction processing:

Hence: **Database must take care of the rollback in case of abort.**

1. b. client can request abort as a programming feature.
2. Either all writes of the transaction, or none become durable: If any write becomes durable, all must become durable.

ACID consistency (high-level)

- a) Is referring to additional high-level features of the DB (and is not part of the basic transaction model we will use) Declarative integrity constraints, such as referential integrity, must hold between transactions.
- b) During the transaction, certain integrity constraints might be violated, but on commit, they must be fulfilled
 - by explicit operations in the transaction,
 - by automatic mechanisms (ON DELETE CASCADE),
 - by user-defined triggers.

Any transaction still violating constraints will be aborted.

ACID Isolation

- Database operations in a transaction appear isolated from database operations of all other transactions
 - does not apply to non-database operations of scripts
- Isolation is expensive:
- May lead to delays, deadlocks, and/or aborts.
 - and can therefore be relaxed: *isolation levels*.
- Highest isolation level:
- Transactions must have same view as with DB-wide MutEx.
- All committed transactions must have the same effect on the database as the execution of the transactions with Mutual exclusion in some order: [Serializability](#).

ACID durability

- Once the user has been notified of success of transaction t :
 - a) All writes of t are permanent: must be changed through later committed writes, e.g. compensating transactions.
 - b) The effect of t is kept in a crash resistant way.
 - minimum requirement: transaction is written to persistent storage: resistant against
 - OS crash
 - system outage
 - c) preferred: protection against loss of persistent memory:
 1. Resistance against persistent storage failure
 2. resistance against catastrophes, geographic distribution

Summary

1. we need transaction demarcation: for Mutex, but also for atomicity: for both **remote** and **concurrent** access.
2. **Transaction**: A sequence of operations that form a logical unit; the client has to request commit.
3. The database needs to roll back failed transactions.
4. DB-wide Mutex delays disjoint-access transaction, does not scale. We need to find a more scalable solution with the same interface as DB-wide Mutex.
5. The ACID Properties (Atomicity, Consistency, Isolation, Durability) are the requirements of transaction processing.